Advanced Neural Network Architectures: Attention Mechanism

Traditional RNNs are powerful for modeling sequential data; however, they struggle with long-term dependencies.

When **sequences become longer**, RNNs struggle to remember information from earlier in the sequence, causing them to underperform on tasks that span many steps.

Attention Mechanism - enables models to learn which part of the sequence to pay most attention to.

CASE: NLP with input sequence of tokens (words, subwords, or characters) mapped to their learned embedding vectors, capturing semantic meaning.

The attention mechanism simultaneously projects each embedding into three different vector representations:

- **Query (Q)**: Represents what the current token is searching for.
- **Key (K)**: Represents what each token can offer to be matched against.
- **Value (V)**: Represents the actual content each token contributes when attended to.

→ **Attention scores**

Each token *attends to every other token in the sequence (including itself)* through the following process (called **self-attention**):

1. Tokens are mapped to their embeddings that contain semantic information.
2. Weight matrices project each embedding into Q, K, and V vectors.
    - Weight matrices `Wq`, `Wk`, and `Wv` are learnable parameters shared across all tokens.

- During training, the weights are optimized for the downstream task (e.g., classification).

3. For each token, we calculate attention scores using the scaled dot product of its Q vector with every token's K vector (including its own):

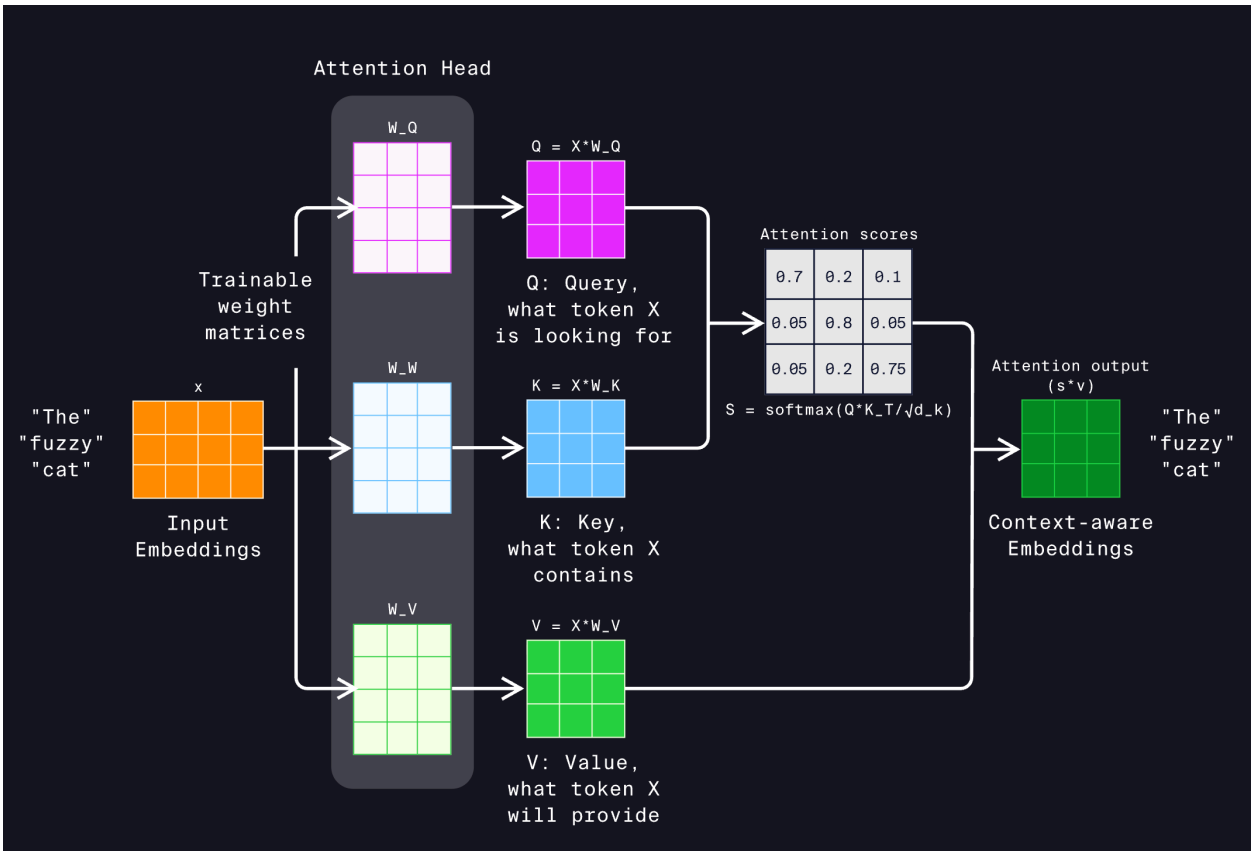
$$attention\ score = \frac{QK^T}{\sqrt{d_k}}$$

4. Normalize the attention scores using the softmax function.
   - This produces attention weights forming a probability distribution over all tokens, with values between 0 and 1.
5. Compute the context vector as a weighted sum of all V vectors, using the normalized attention weights.
   - The context vector encodes the most relevant information from the sequence based on what the current token attends to.
   - This output enriches the original embeddings with contextual information from the entire sequence.

Q, K, and V → Search system that helps the model determine which parts of the sequence are most relevant for understanding each token.

Here's an example of computing attention for the token `"cat"` in the sentence `["The", "fuzzy", "cat"]`:

Token	Unnormalized Attention Score	Normalized Attention Weight
The	-0.222	0.165
fuzzy	0.181	0.247
cat	0.027	0.211



Attention Mechanism Implementation

Scaled Dot-Product Self-Attention

Tokenized Sequence:

```
["the", "fuzzy", "cat"]
```

Each token has been mapped to a pre-trained 8-dimensional embedding

As an example, the token `"cat"` might have the embedding vector: `[-0.3800, 0.3322, -0.4134, -0.4029, -0.0850, -0.3236, -0.3624, -0.4108]`.

We project each embedding into its respective Q, K, and V vectors using weight matrices created as linear layers without bias parameters:

```
import math
import torch
import torch.nn.functional as F
```

```
d = 8  # Embedding size
Wq = torch.nn.Linear(d, d, bias=False)
```

```
Wk = torch.nn.Linear(d, d, bias=False)
Wv = torch.nn.Linear(d, d, bias=False)
```

Each projection has shape [T, d]

T - number of tokens

d - embedding dimension

Then we calculate attention scores for each token using its Q vector compared against the K vectors of all tokens:

```
scores = Q @ K.transpose(-2, -1) / math.sqrt(d)
```

The scores are normalized using the **softmax** function:

```
weights = F.softmax(scores, dim=-1)
```

0 - 1 range

Then: we create context representations by computing the weighted sum of V vectors:

```
context = weights @ V
```

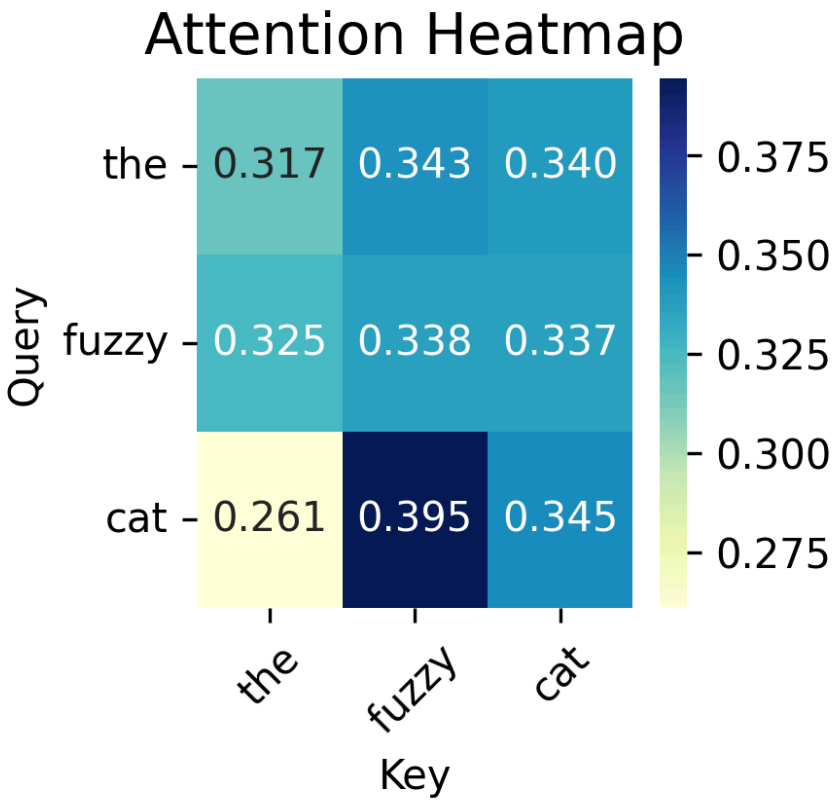
Output: [T, d]

```
weights_np = weights.detach().cpu().numpy()
T = len(tokens)
```

```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(T, T))
sns.heatmap(weights_np,
            xticklabels=tokens,
            yticklabels=tokens,
            cmap="YlGnBu",
            annot=True,
            fmt=".3f",
            cbar=True)

plt.xlabel("Key")
plt.ylabel("Query")
plt.title("Attention Heatmap")
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.show()
```


Attention Heatmap
the
fuzzy
cat
Query
0.317
0.343
0.340
0.325
0.338
0.337
0.261
0.395
0.345
0.375
0.350
0.325
0.300
0.275
the
fuzzy
cat
Key